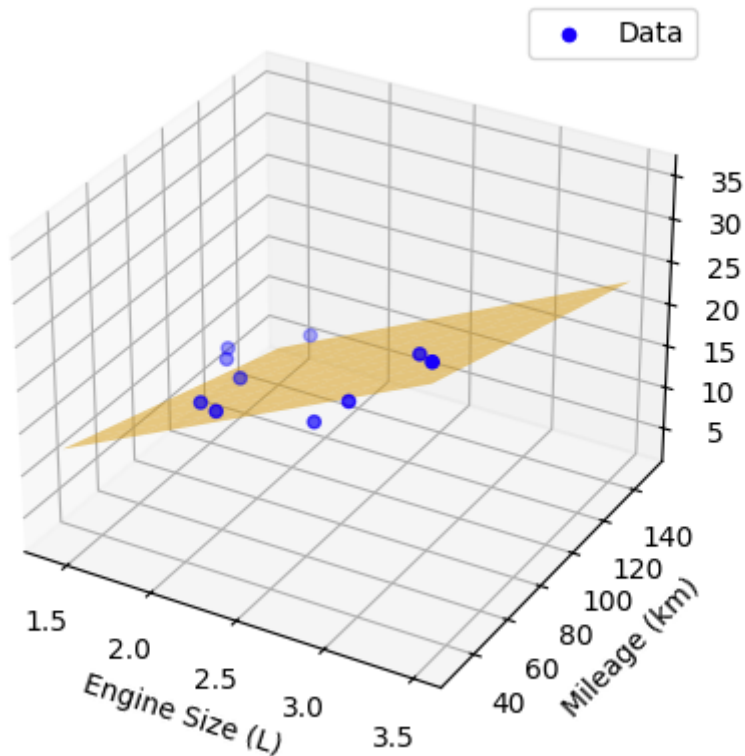


Question 1: Price and Speed of Sale Prediction

A. Linear Regression: Price Prediction

Q1.A.2: Visualization

Linear Regression: Price Prediction

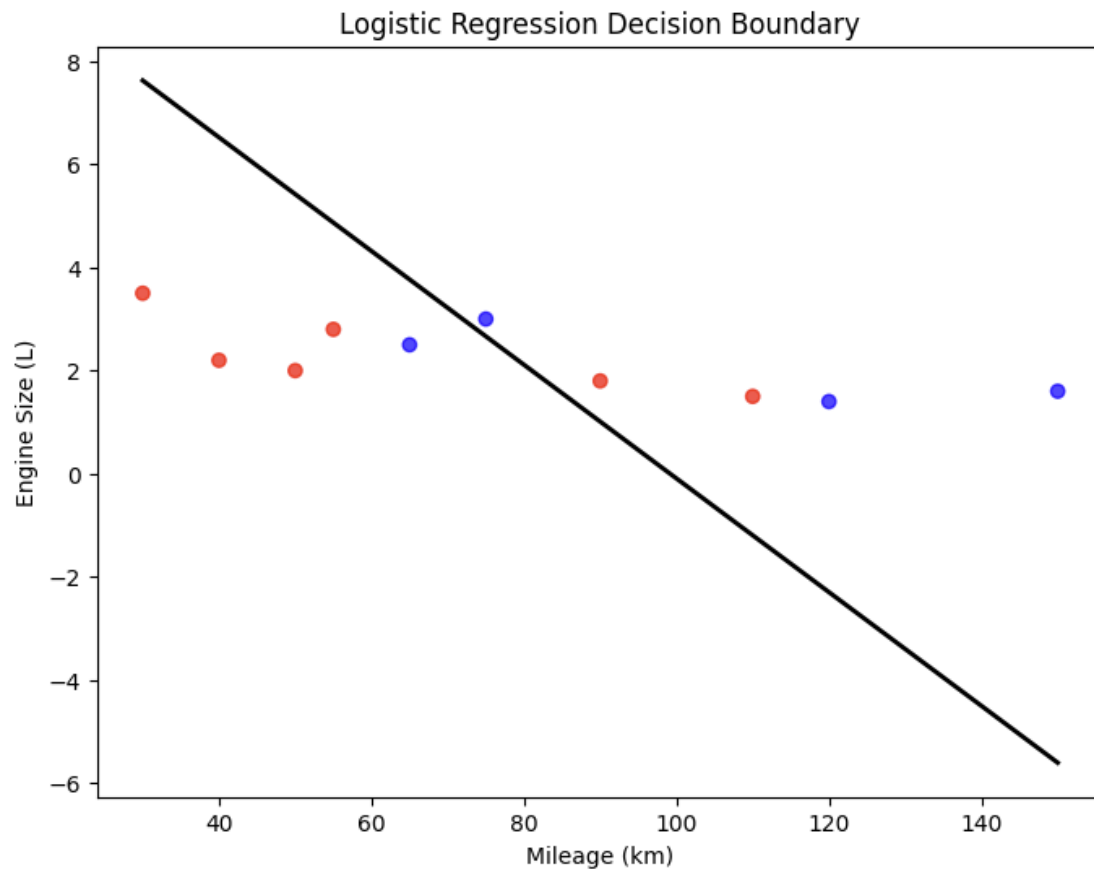


Q1.A.3:

Cars with bigger engines cost more. Cars that have been driven a lot cost less. The model draws a plane that tries to guess the price for each car. It's not perfect – but it gives a rough idea of how price relates to engine size and mileage.

B. Logistic Regression: Quick Sale Classification

Q1.B.2:



Q1.B.3:

The scatterplot shows the cars with Mileage on the x-axis and Engine Size on the y-axis, colored by actual outcome (Y = sold quickly, N = not). The black line is the model's 50% decision boundary. Points below the line are predicted to sell quickly, points above are predicted not to sell.

Looking at the plot, a blue point above the line and/or a red point below the line indicate misclassifications. These match the confusion matrix.

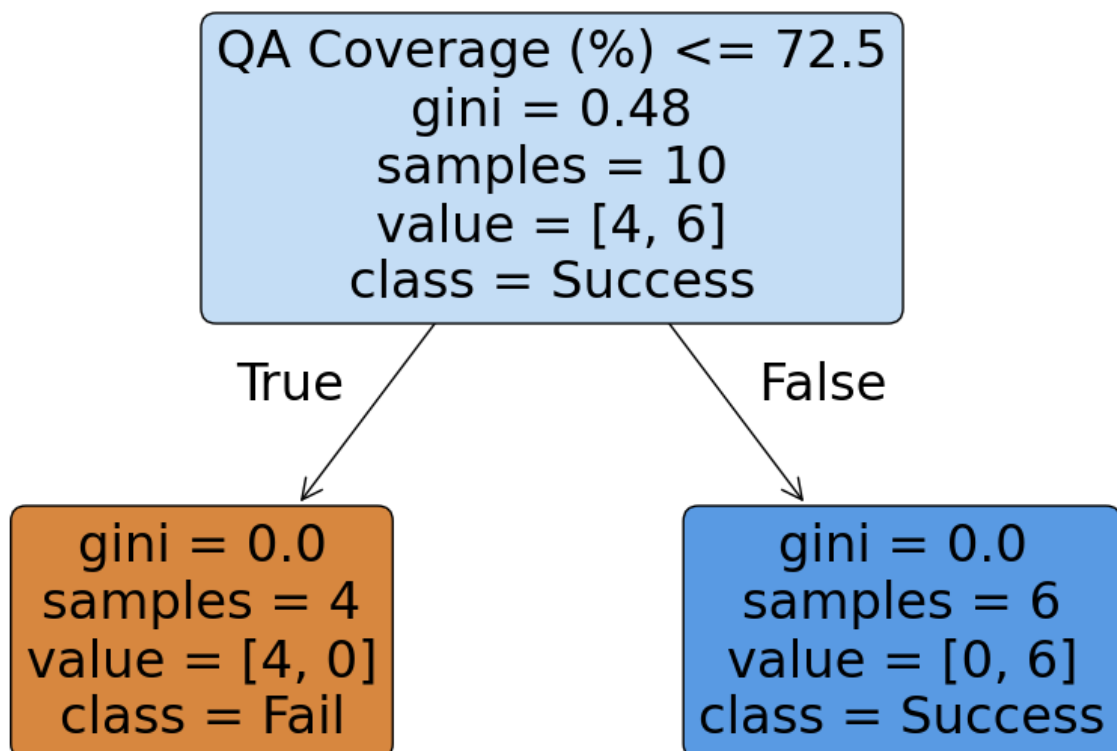
Question 2: The Logic of Failure

The "Failure Pathway" Decision Tree

Q2.A.2: Failure node: 1, samples: 4, failure ratio: 1.00

Q2.A.3:

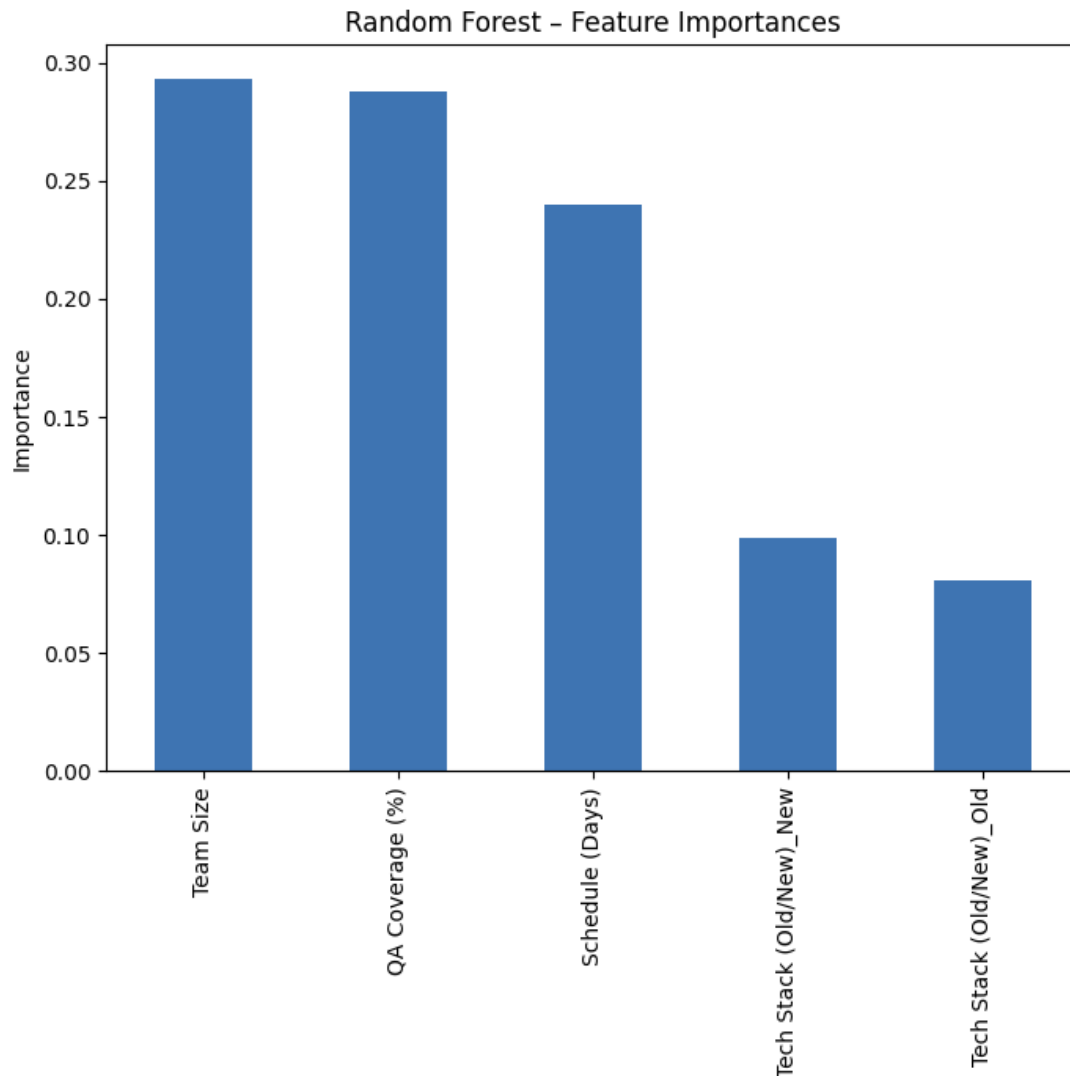
Decision Tree - Software Project Success



Project Management Rule "Never start a project with planned QA Coverage below 73 %. All four projects with QA $\leq$ 72.5 % failed."

B. The "Black Box" Consensus

Q2.B.3: Random Forest Feature Importances:



Q2.B.4: Hypothesis & Defence

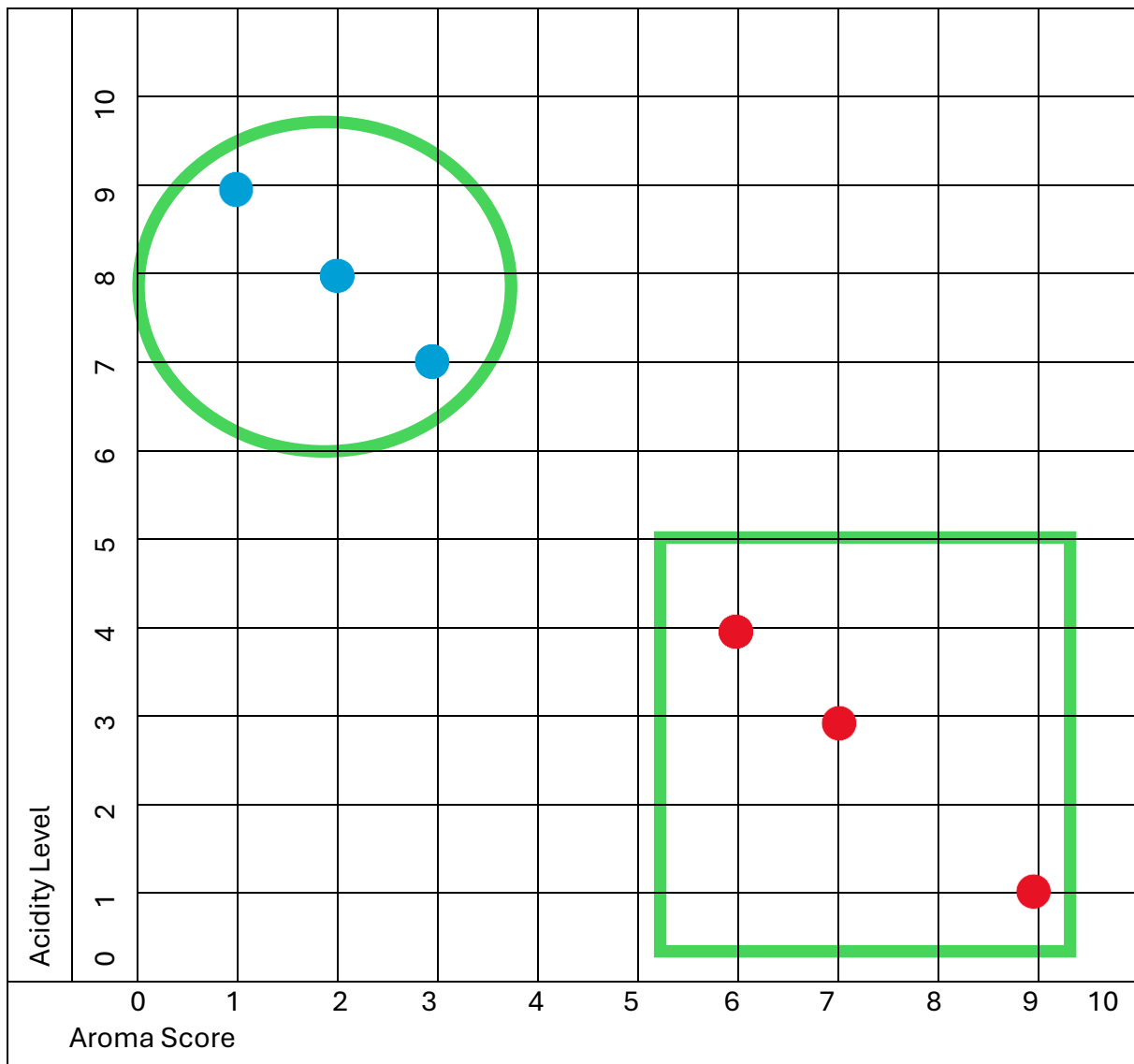
The Random Forest ranks **Team Size** and **QA Coverage (%)** as the most important factors for predicting project success. This suggests that both staffing and testing coverage really matter.

Interestingly, Team Size being the top factor fits the old saying: “too many cooks spoil the broth.” Bigger teams often bring more coordination issues, dependencies, and miscommunication risks.

Looking at it this way, my choice to work alone might have been a surprisingly strategic move — at least according to the model.

Question 3 - The K-Means Balancing Act

A. Initial Setup



B. Iteration 1

$$d(P1, CA) = \sqrt{(1 - 1)^2 + (9 - 9)^2} = 0$$

$$d(P1, CB) = \sqrt{(1 - 9)^2 + (9 - 1)^2} = \sqrt{128} \approx 11.3$$

In the same way, the distances for the remaining points are calculated (and yes, I've used AI for the calculations). The results are summarized in the table below.

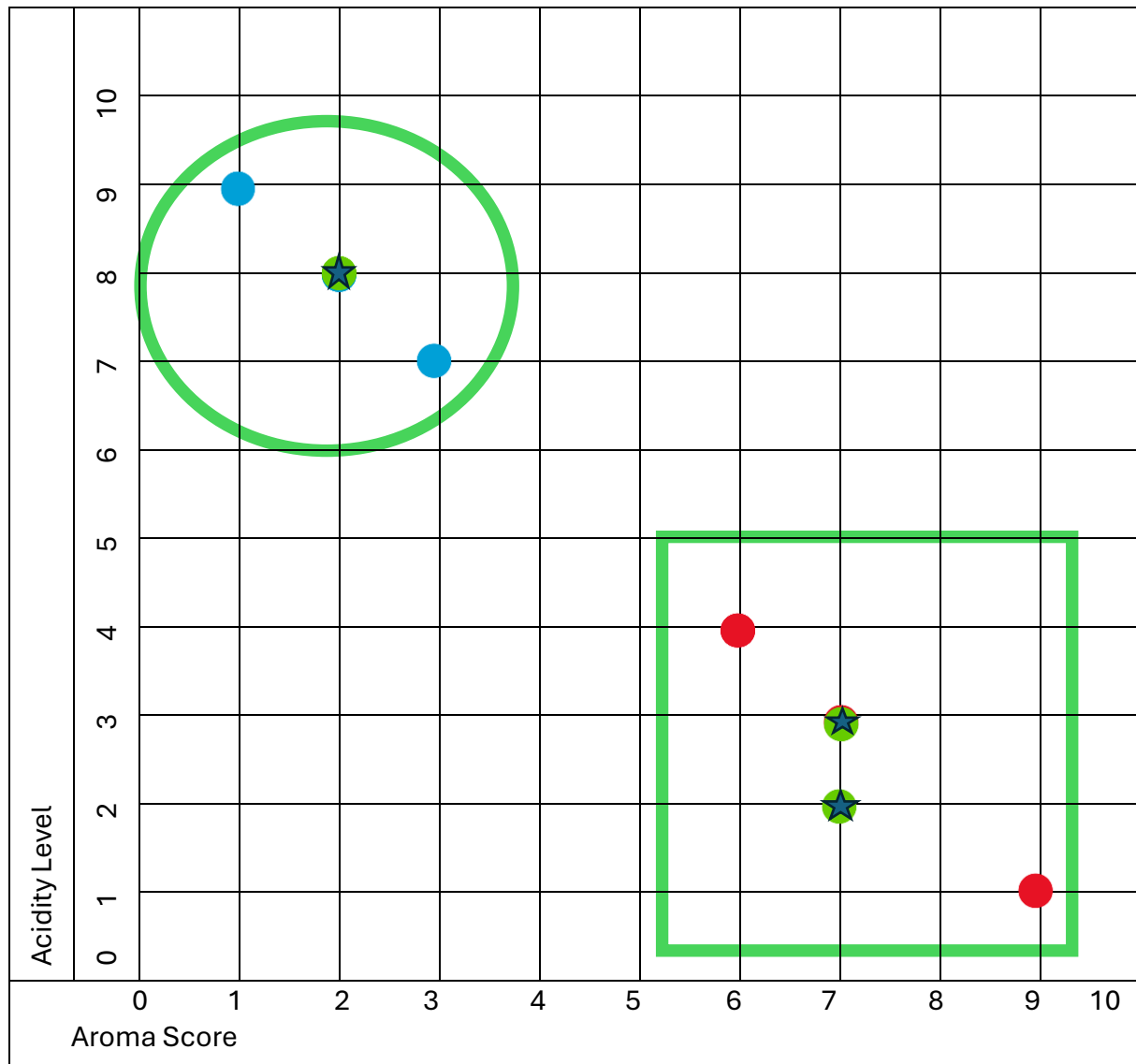
Point	Coordinates	Dist to CA (1,9)	Dist to CB (9,1)	Assigned Cluster
P1	(1,9)	0.0	11.3	A
P2	(2,8)	1.4	9.9	A
P3	(3,7)	2.8	8.5	A
P4	(6,4)	6.4	5.0	B
P5	(7,3)	7.8	2.8	B
P6	(9,1)	11.3	0.0	B

Q3.B.3: Update Step:

$$\text{CA}_{\text{new}} = \left(\frac{1+2+3}{3}, \frac{9+8+7}{3} \right) = (2,8)$$

$$\text{CB}_{\text{new}} = \left(\frac{6+7+9}{3}, \frac{4+3+1}{3} \right) = (7.3, 2.7)$$

C. Iteration 2



Q3.D.1

After updating the centroids in Iteration 2, none of the points changed clusters. This means the algorithm has converged, and the final cluster assignments are stable. The points stayed in the same regions as in the initial plot, showing that the clustering is done.

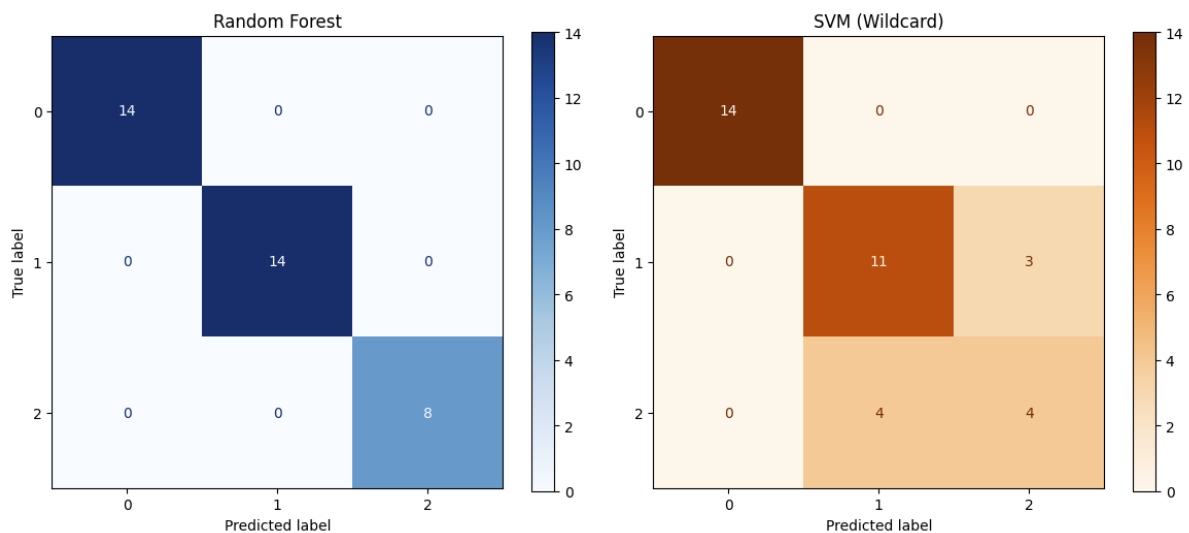
Q3.D.2

- **Cluster A:** Beans with **bright acidity and subtle aroma** — likely lighter roasts or fruity origins.
- **Cluster B:** Beans with **milder acidity and stronger aroma** — possibly darker roasts or more full-bodied varieties.

Question 4 - Uncorking the Data (Code Submission)

Q4.B.4 Defence

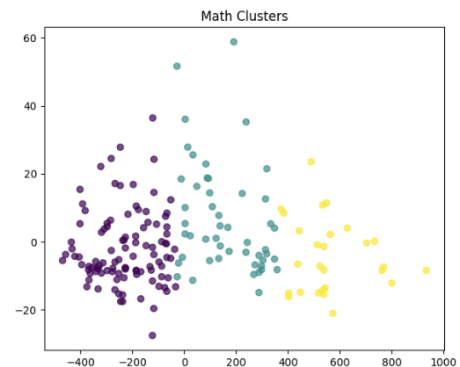
- a) Looking at the confusion matrices, the Random Forest did a better job. It got all the test samples right for all three wine classes (0, 1, 2), so the accuracy was perfect. The SVM, on the other hand, got a few samples wrong in classes 1 and 2, often mixing up class 2 with class 1.
- b) I googled scikit-learn wine and came across this page: <https://github.com/Joseph89155/wine-classification-ml>, where I read that 'Random Forest and SVM showed the highest performance on the test set.' Since I had already used Random Forest, I decided to try SVM



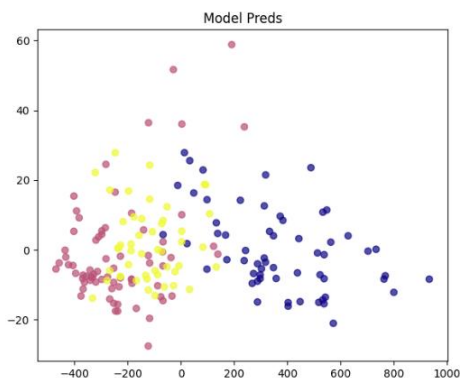
Q4.C.3 Analysis & Defence

a) Defending the Unsupervised Result:

Looking at Subplot 1 (K-Means), the algorithm grouped wines based on distances in PCA space. Compared to Subplot 3 (actual cultivars), it's clear that Class 2 is spread out and overlaps with other classes, so K-Means ended up splitting it across clusters. This happens because K-Means assumes clusters are roughly spherical and evenly sized – which the real data isn't.



b) Defending the Supervised Result:



Subplot 2 (Random Forest predictions) shows that the supervised model handled the tricky edge cases better than K-Means. Since it learned from labeled data, it could separate points in overlapping areas using multiple decision rules. That's why it got the boundary cases between Class 1 and Class 2 mostly right, whereas K-Means struggled.

c) Ambiguity Check:

In the center of the PCA plot, K-Means and the supervised predictions disagree on a few points. K-Means put some points in Cluster 1 that actually belong to Class 2, while the supervised model classified them correctly. So in that ambiguous region, the supervised model is clearly closer to the ground truth.

